

CSE3001 Software Engineering Module-III System Model

System models

- Abstract descriptions of systems whose requirements are being analysed

System modelling

- System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers
- Different models present the system from different perspectives
 - Structural perspective showing the system or data architecture
 - External perspective / Function perspective showing the system's context or environment
 - Behavioural perspective showing the behaviour of the system

Structured methods

- Structured methods incorporate system modelling as an inherent part of the method
- Methods define a set of models, a process for deriving these models and rules and guidelines that should apply to the models
- CASE tools support system modelling as part of a structured method

Method weaknesses

- They do not model non-functional system requirements
- They do not usually include information about whether a method is appropriate for a given problem
- They may produce too much documentation
- The system models are sometimes too detailed and difficult for users to understand

Model types

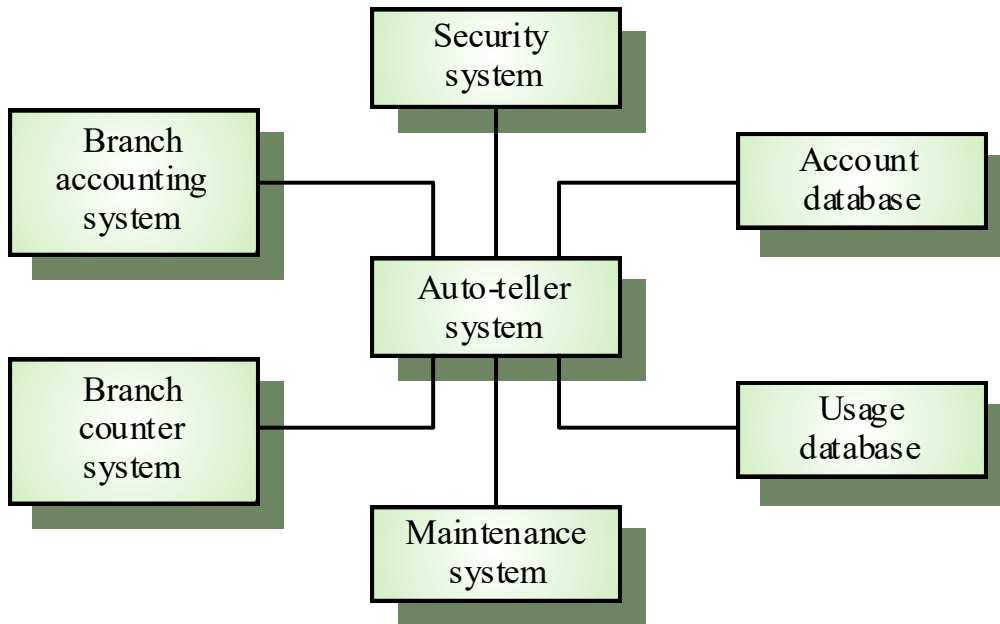
- Data processing model showing how the data is processed at different stages
- Composition model showing how entities are composed of other entities
- Architectural model showing principal sub-systems
- Classification model showing how entities have common characteristics
- Stimulus/response model showing the system's reaction to events

Context models

- Context models are used to illustrate the boundaries of a system
- Social and organisational concerns may affect the decision on where to position system boundaries

- Architectural models show the a system and its relationship with other systems

The context of an ATM system



Process models

- Process models show the overall process and the processes that are supported by the system
- Data flow models may be used to show the processes and the flow of information from one process to another

Data-processing models

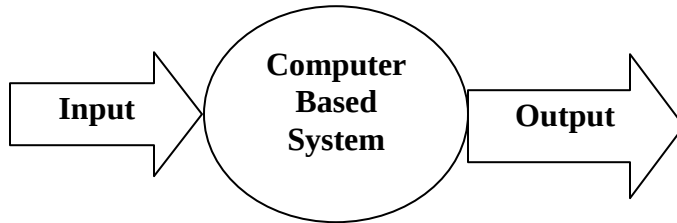
- Data flow diagrams are used to model the system's data processing
- These show the processing steps as data flows through a system
- Intrinsic part of many analysis methods
- Simple and intuitive notation that customers can understand
- Show end-to-end processing of data

Data Processing Diagram - Guidelines

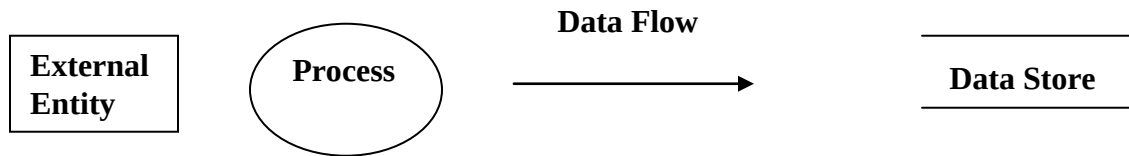
- All icons must be labeled with meaningful names
- The DFD evolves through A number of levels of detail
- Always begin with A context level diagram (also called level 0)
- Always show external entities at level 0
- Always label data flow arrows
- Do not represent procedural logic

The Process Model

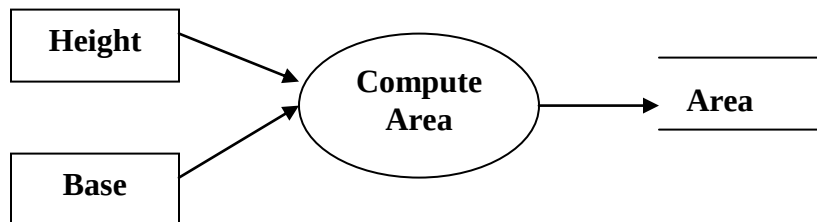
Every computer-based system is an information transform



Process Flow Modeling Notation



Example:



Constructing a DFD

- Review the data model to isolate data objects and use a grammatical parse to determine “operations”
- Determine external entities (producers and consumers of data)
- Create a level 0 DFD

Constructing a DFD

- Write a narrative describing the transform
- Parse to determine next level transforms
- “Balance” the flow to maintain data flow continuity
- Develop a level 1 DFD
- Use a 1:5 (approx.) Expansion ratio

Flow Modeling Notes

- Each bubble is refined until it does just one thing
- The expansion ratio decreases as the number of levels increase

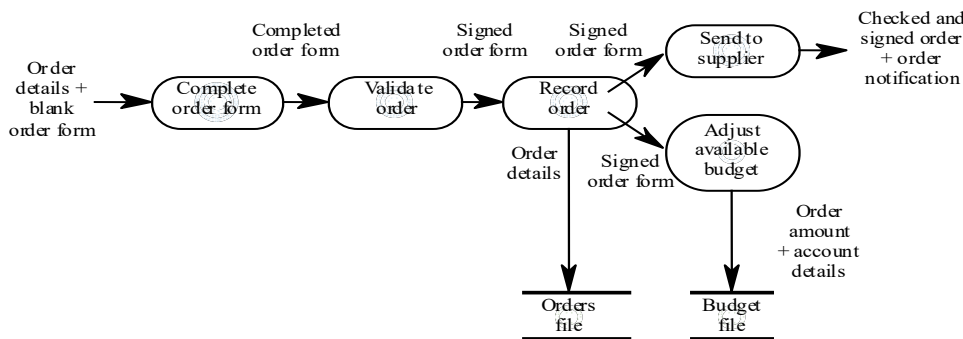
- Most systems require between 3 and 7 levels for an adequate flow model

A single data flow item (arrow) may be expanded as levels increase (data dictionary provides information)

Level 0 DFD Example



Order processing DFD



Behavioural models

- Behavioural models are used to describe the overall behaviour of a system
- Two types of behavioural model are shown here
 - Data processing models that show how data is processed as it moves through the system
 - State machine models that show the systems response to events
- Both of these models are required for a description of the system's behaviour

Behavioral Modeling

- The behavioral model indicates how software will respond to external events or stimuli. To create the model, the analyst must perform the following steps:

- Evaluate all use-cases to fully understand the sequence of interaction within the system.
- Identify events that drive the interaction sequence and understand how these events relate to specific objects.
- Create a sequence for each use-case.
- Build a state diagram for the system.
- Review the behavioral model to verify accuracy and consistency.

State machine models

- These model the behaviour of the system in response to external and internal events
- They show the system's responses to stimuli so are often used for modelling real-time systems
- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another

State Representations

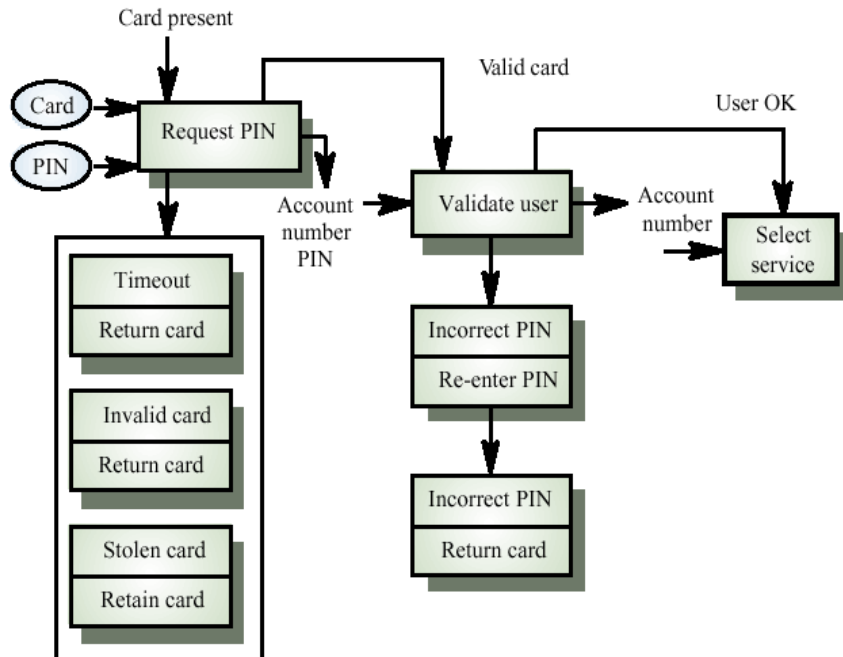
- In the context of behavioral modeling, two different characterizations of states must be considered:
 - The state of each class as the system performs its function and
 - The state of the system as observed from the outside as the system performs its function
- The state of a class takes on both passive and active characteristics.
 - A *passive state* is simply the current status of all of an object's attributes.
 - The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing.

The States of a System

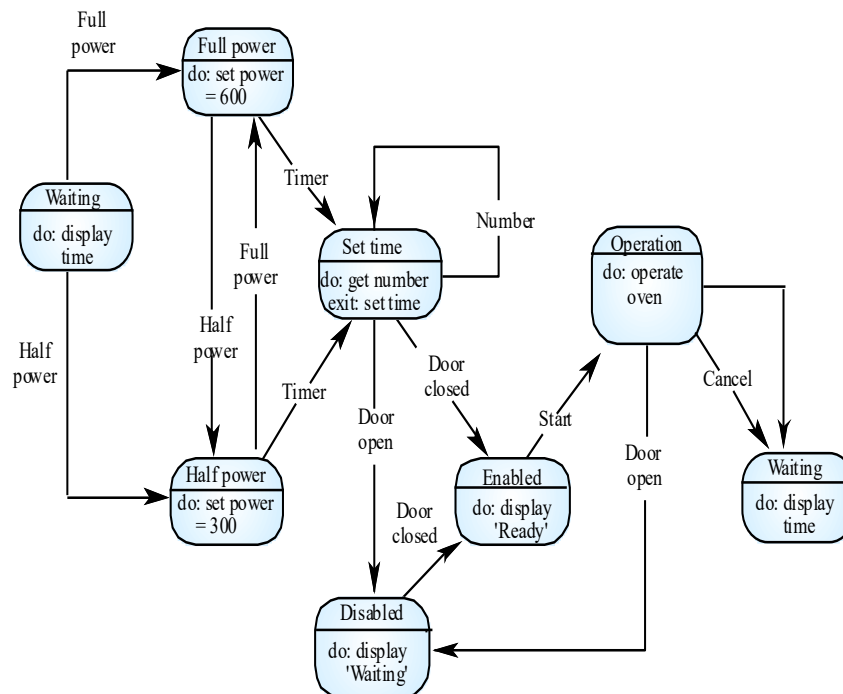
- *State*: A set of observable circumstances that characterizes the behavior of a system at a given time
- *State transition*: The movement from one state to another
- *Event*: An occurrence that causes the system to exhibit some predictable form of behavior
- *Action*: Process that occurs as a consequence of making a transition.

Example 1: ATM Transaction

System Model



Example -2 Microwave Oven



Microwave oven state description

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for 5 seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

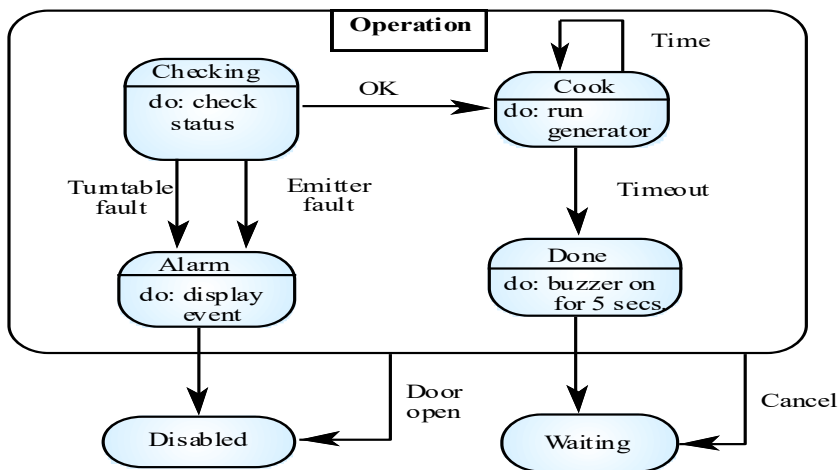
Microwave oven stimuli

Stimulus	Description
Half power	The user has pressed the half power button
Full power	The user has pressed the full power button
Timer	The user has pressed one of the timer buttons
Number	The user has pressed a numeric key
Door open	The oven door switch is not closed
Door closed	The oven door switch is closed
Start	The user has pressed the start button
Cancel	The user has pressed the cancel button

State charts

- Allow the decomposition of a model into sub-models.
- A brief description of the actions is included following the 'do' in each state.
- Can be complemented by tables describing the states and the stimuli.

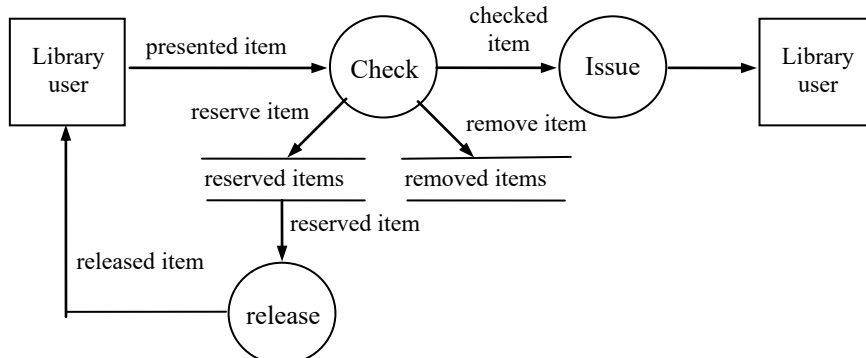
Microwave oven operation



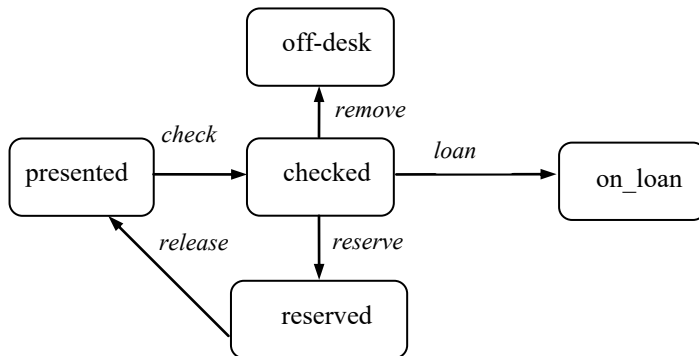
Example:

- Consider a library item presented the user at the issue desk for borrowing, returning or reserving
- 'Library world' can be partitioned into the domains of the issue desk and the library user
- Data-flow and state transition schemes are used to model the library item from point of view each domain

Data-flow model -Issue desk domain



State transition model -Issue Desk Domain



Semantic Data Models

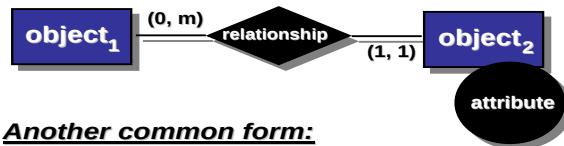
- Used to describe the logical structure of data processed by the system
- Entity-relation-attribute model sets out the entities in the system, the relationships between these entities and the entity attributes
- Widely used in database design. Can readily be implemented using relational databases

Building an ERD

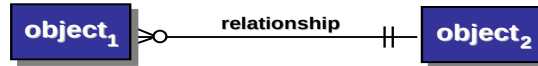
- Level 1: Model all data objects (entities) and their “connections” to one another.
- Level 2: Model all entities and relationships.
- Level 3: Model all entities, relationships, and the attributes that provide further depth.

ERD Notation

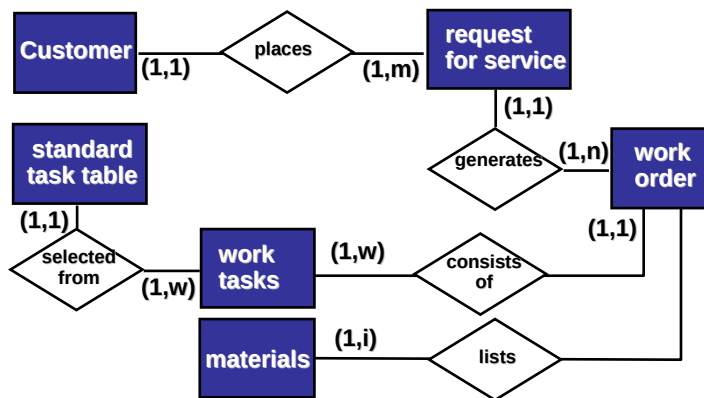
One common form:



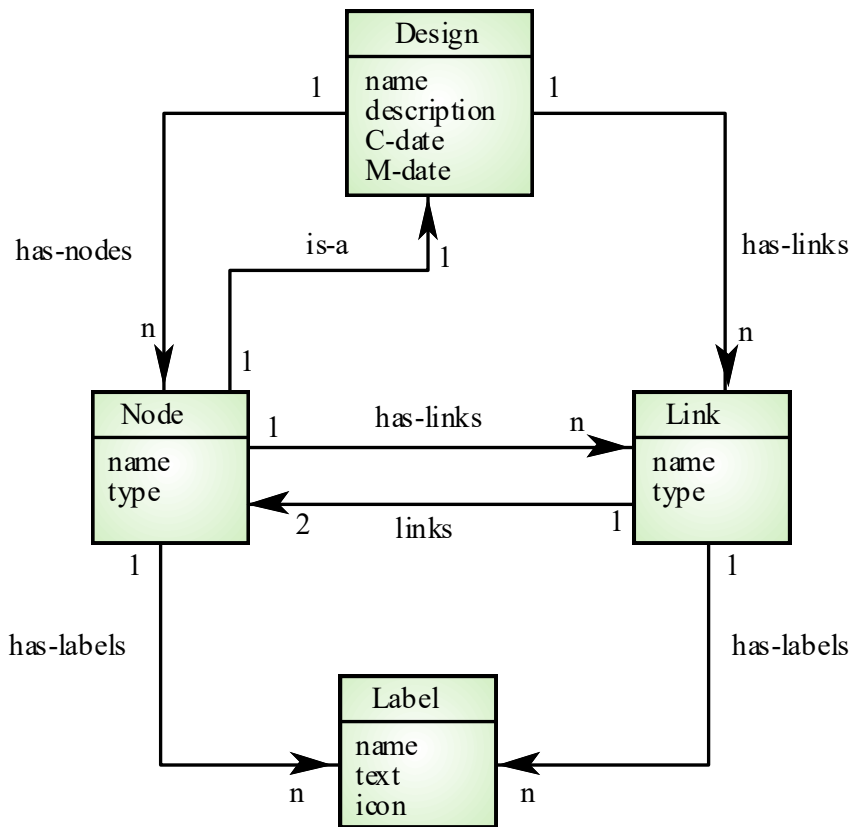
Another common form:



ERD: An Example



Example: Software Design Semantic Model



Data Dictionaries

- Data dictionaries are lists of all of the names used in the system models. Descriptions of the entities, relationships and attributes are also included
- Advantages
 - Support name management and avoid duplication
 - Store of organisational knowledge linking analysis, design and implementation
- Many CASE workbenches support data dictionaries

Data Dictionary Entries

Name	Description	Type
has-labels	1:N relation between entities of type Node or Link and entities of type Label.	Relation
Label	Holds structured or unstructured information about nodes or links. Labels are represented by an icon (which can be a transparent box) and associated text.	Entity
Link	A 1:1 relation between design entities represented as nodes. Links are typed and may be named.	Relation
name (label)	Each label has a name which identifies the type of label. The name must be unique within the set of label types used in a design.	Attribute
name (node)	Each node has a name which must be unique within a design. The name may be up to 64 characters long.	Attribute

Object Models

- Object models describe the system in terms of object classes
- An object class is an abstraction over a set of objects with common attributes and the services (operations) provided by each object
- Various object models may be produced
 - Inheritance models
 - Aggregation models
 - Interaction models
- Natural ways of reflecting the real-world entities manipulated by the system
- More abstract entities are more difficult to model using this approach
- Object class identification is recognised as a difficult process requiring a deep understanding of the application domain
- Object classes reflecting domain entities are reusable across systems

Data Object

- Described by set of attributes (data items) and will be manipulated within the software (system).

Types of Object

External entities (printer, user, sensor)

Things (e.g., reports, displays, signals)

Occurrences or events (e.g., interrupt, alarm)

Roles (e.g., manager, engineer, salesperson)

Organizational units (e.g., division, team)

Places (e.g., manufacturing floor)

Structures (e.g., employee record)

Data Objects and Attributes

A data object contains a set of attributes that act as an aspect, quality, characteristic, or descriptor of the object

object: automobile

attributes:

make

model

body type

price

options code

Relationship?

- Several instances of a relationship can exist
- Objects can be related in many different ways

Object Modeling

- Must be understood to apply class-based elements of the analysis model
- Key concepts:
 - Classes and objects
 - Attributes and operations
 - Encapsulation and instantiation
 - Inheritance

Classes

- Object-oriented model begins with the definition of a class, often defined as:
 - Template
 - Generalized description
 - “Blueprint” describing a collection of similar items
- A meta-class (also called a super class) establishes a hierarchy of classes.
- Once a class of items is defined, a specific instance of the class can be identified.

Class-Based Modeling

- Identify analysis classes by examining the problem statement
- Use a “grammatical parse” to isolate potential classes
- Identify the attributes of each class
- Identify operations that manipulate the attributes

Analysis Classes

- *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- *Things* (e.g, reports, displays, letters, signals) that are part of the information domain for the problem.
- *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
- *Organizational units* (e.g., division, group, team) that are relevant to an application.
- *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

Selecting Classes—Criteria

Retained information

Needed services

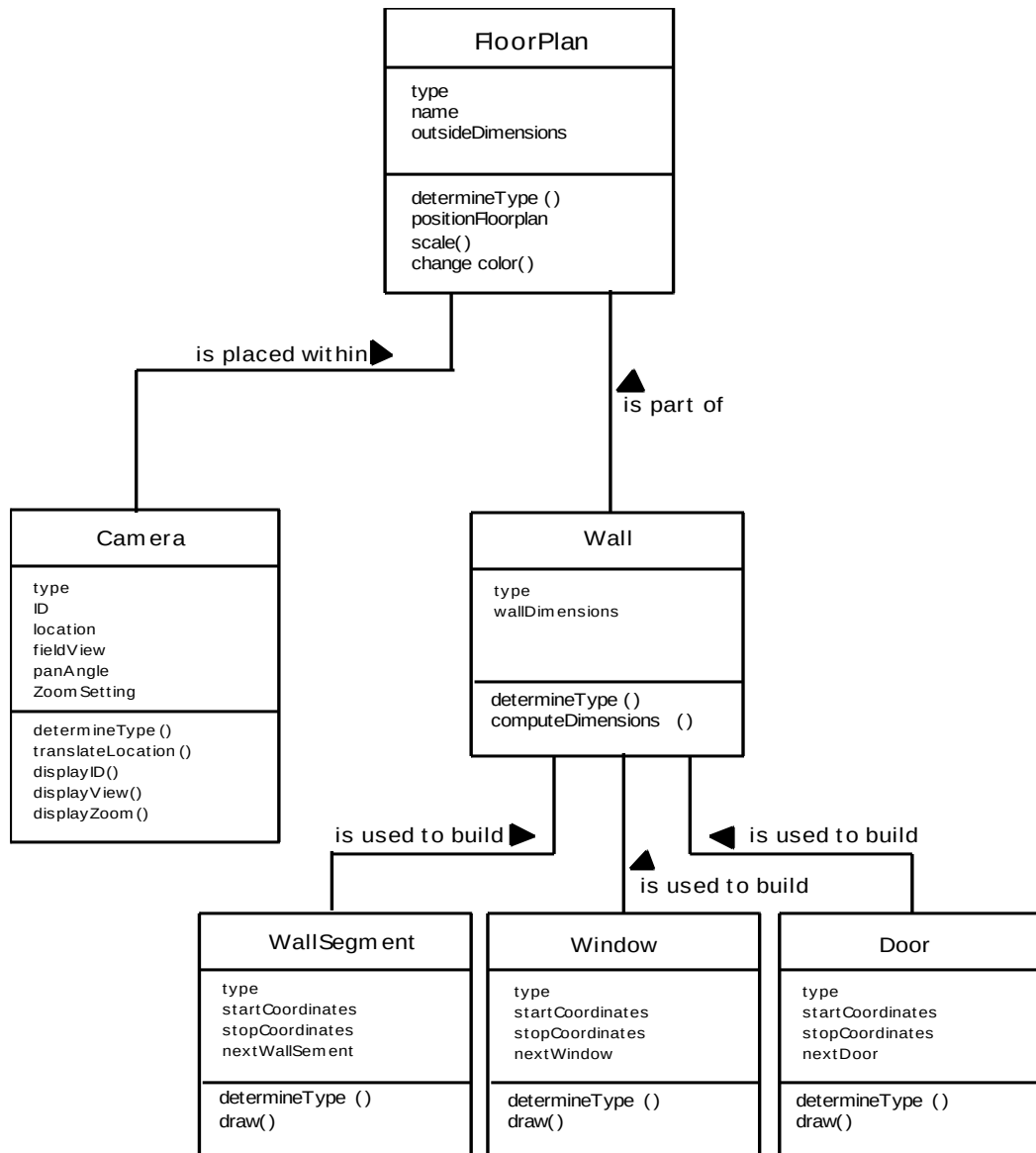
Multiple attributes

Common attributes

Common operations

Essential requirements

Class Diagram



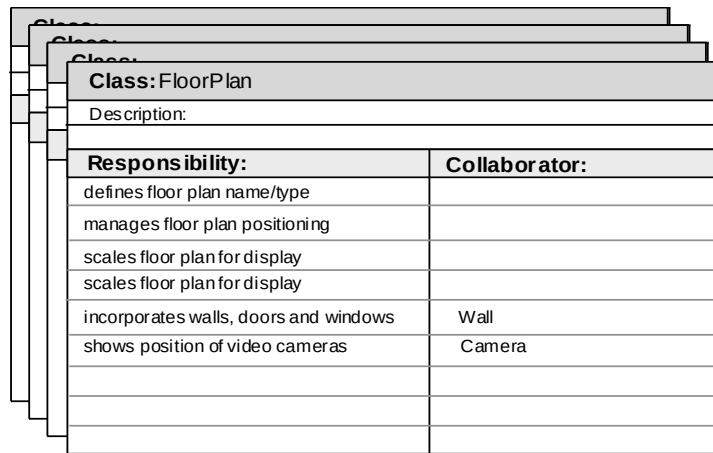
CRC Modeling

- Analysis classes have “responsibilities”
 - *Responsibilities* are the attributes and operations encapsulated by the class
- Analysis classes collaborate with one another
 - *Collaborators* are those classes that are required to provide a class with the information needed to complete a responsibility.
 - In general, collaboration implies either a request for information or a request for some action.

Class Types

- *Entity* classes, are directly from of the FloorPlan
- *Boundary* used to create (e.g., screen or reports) that and interacts software is used.

CRC Modeling



Class: FloorPlan	
Description:	
Responsibility:	Collaborator:
defines floor plan name/type	
manages floor plan positioning	
scales floor plan for display	
scales floor plan for display	
incorporates walls, doors and windows	Wall
shows position of video cameras	Camera

classes, also or business extracted the statement problem (e.g., and Sensor). classes are the interface interactive printed the user sees with as the

- *Controller classes* manage a “unit of work” from start to finish. That is, controller classes can be designed to manage
 - The creation or update of entity objects;
 - The instantiation of boundary objects as they obtain information from entity objects;
 - Complex communication between sets of objects;
 - Validation of data communicated between objects or between the user and the application.

Responsibilities

- System intelligence should be distributed across classes to best address the needs of the problem
- Each responsibility should be stated as generally as possible
- Information and the behavior related to it should reside within the same class
- Information about one thing should be localized with a single class, not distributed across multiple classes.
- Responsibilities should be shared among related classes, when appropriate.

Collaborations

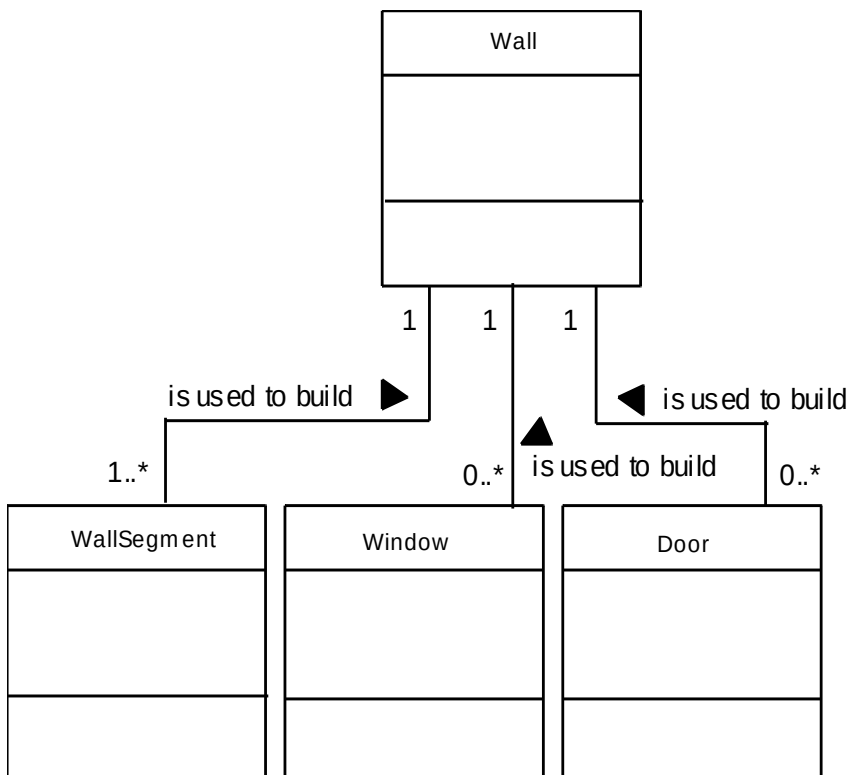
- Classes fulfill their responsibilities in one of two ways:
 - A class can use its own operations to manipulate its own attributes, thereby fulfilling a particular responsibility, or

- A class can collaborate with other classes.
- Collaborations identify relationships between classes
- Collaborations are identified by determining whether a class can fulfill each responsibility itself
- Three different generic relationships between classes
 - The *is-part-of* relationship
 - The *has-knowledge-of* relationship
 - The *depends-upon* relationship

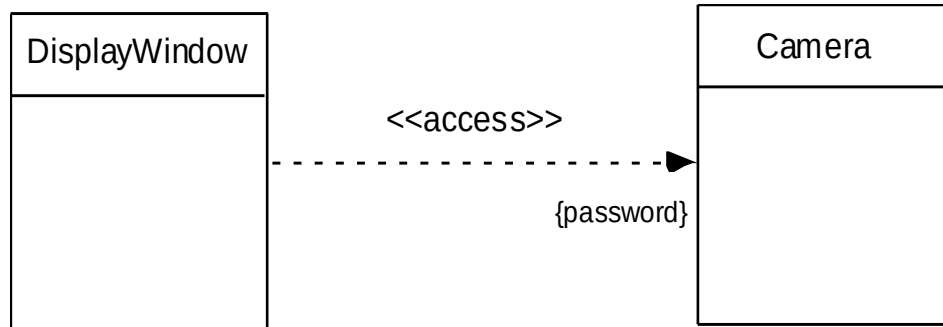
Associations and Dependencies

- Two classes are often related to one another in some fashion
 - *associations*
 - Associations can be refined by indicating *multiplicity* (the term *cardinality* is used in data modeling)
- In many instances, a client-server relationship exists between two classes.
 - In such cases, a client-class depends on the server-class in some way and a *dependency relationship* is established

Multiplicity



Dependencies

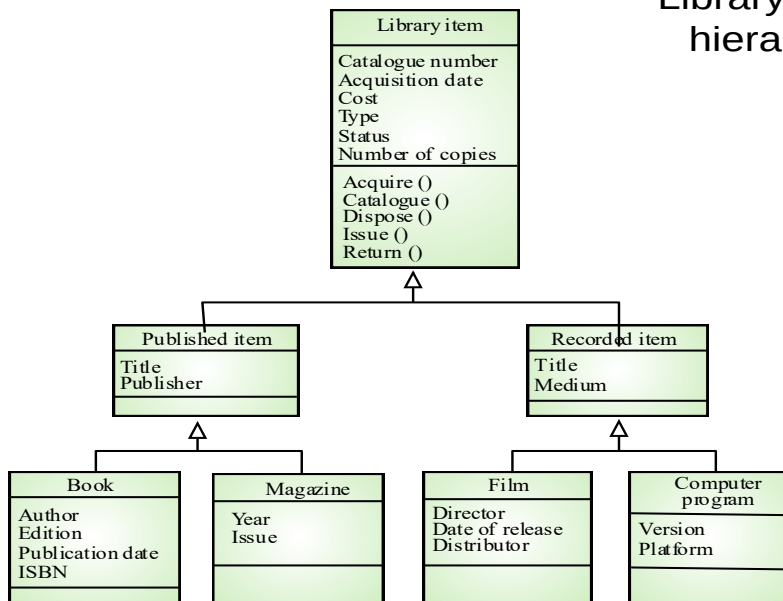


Inheritance

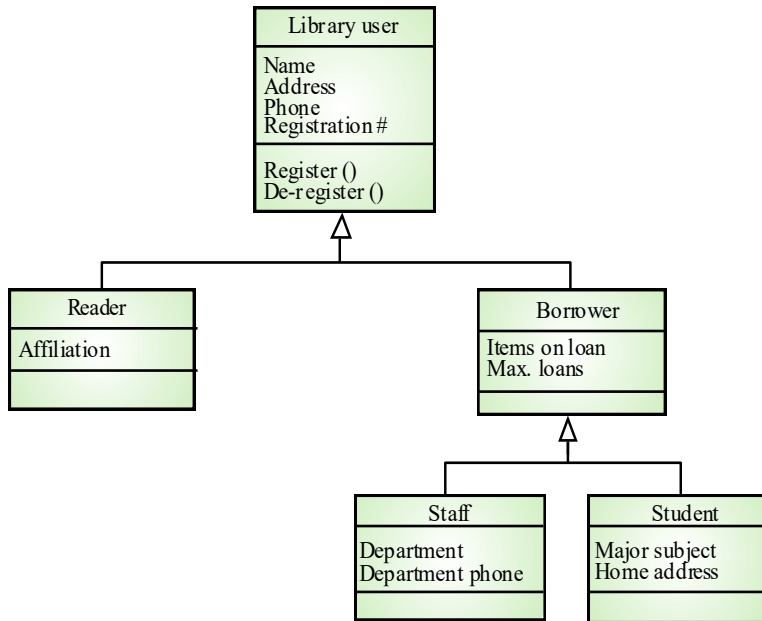
- Organise the object hierarchy
- Classes at the top of the hierarchy reflect the common features of all classes
- Object classes inherit their attributes and services from one or more super-classes.
- Class hierarchy design is a difficult process if duplication in different branches is to be avoided

models domain classes into a

Library class hierarchy

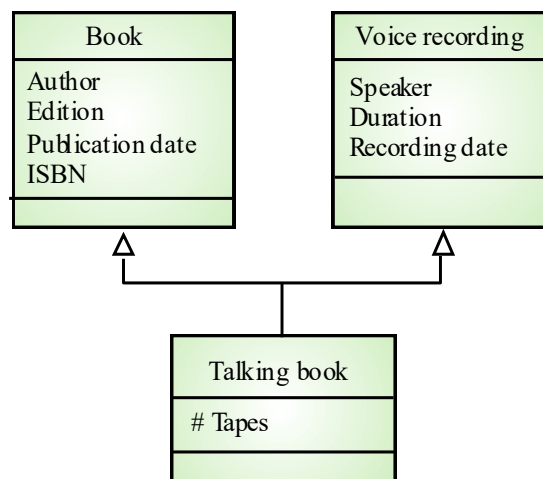


User Class Hierarchy



Multiple inheritance

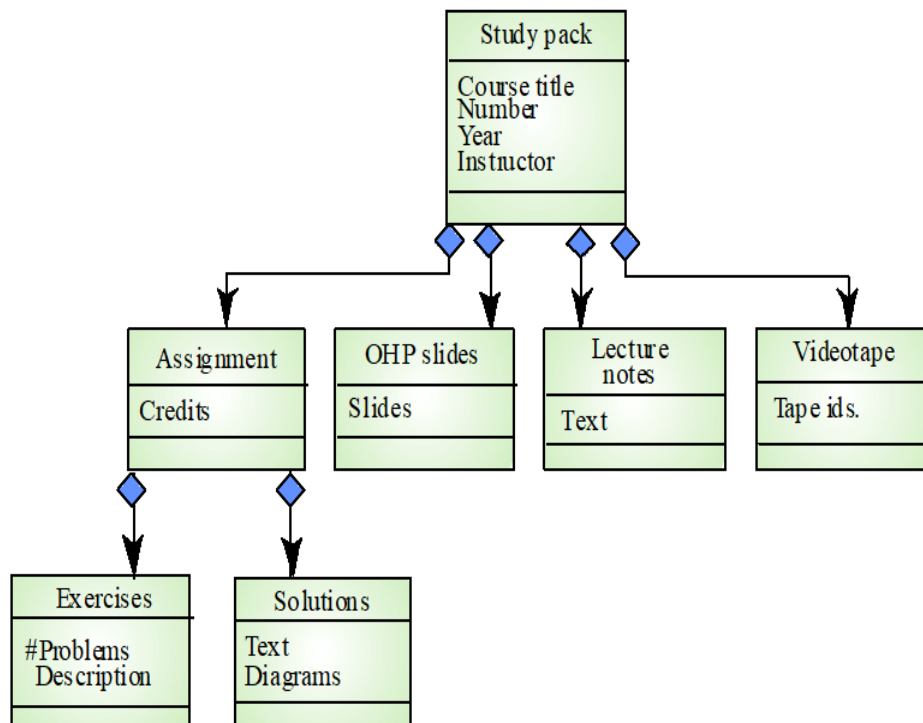
- A system which supports multiple inheritance allows object classes to inherit from several super-classes
- Can lead to semantic conflicts where attributes/services with the same name in different super-classes have different semantics
- Makes class hierarchy reorganisation more complex.



Object aggregation

- Aggregation model shows how classes which are collections are composed of other classes
- Similar to the *part-of* relationship in semantic data models

Object aggregation

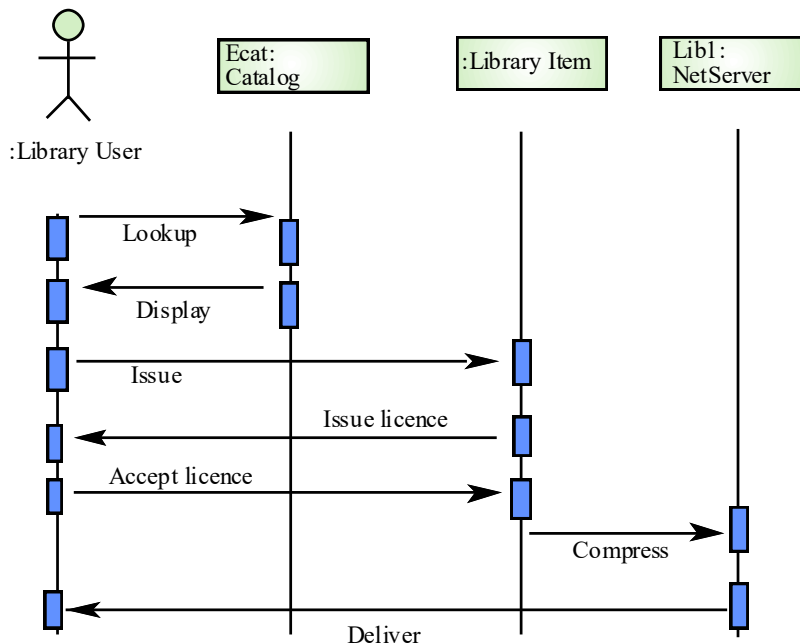


Object

behaviour modelling

- A behavioural model shows the interactions between objects to produce some particular system behaviour that is specified as a use-case
- Sequence diagrams (or collaboration diagrams) are used to model interaction between objects

Example: Issue of electronic items



Scenarios

- Scenarios are descriptions of how a system is used in practice
- They are helpful in requirements elicitation as people can relate to these more readily than abstract statement of what they require from a system
- Scenarios are particularly useful for adding detail to an outline requirements description

Scenario descriptions

- System state at the beginning of the scenario
- Normal flow of events in the scenario
- What can go wrong and how this is handled
- Other concurrent activities
- System state on completion of the scenario

Event scenarios

- Event scenarios may be used to describe how a system responds to the occurrence of some particular event.
- A diagrammatic convention for event scenarios.
 - Data provided and delivered

- Control information
- Exception processing
- The next expected event

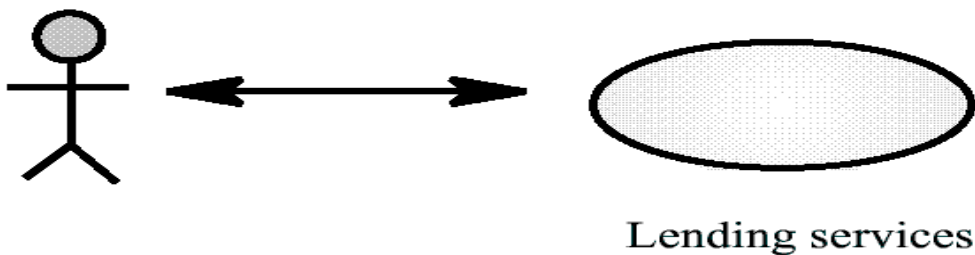
Use-Cases

- A scenario that describes a “thread of usage” for a system
- *Actors* represent roles people or devices play as the system functions
- *Users* can play a number of different roles for a given scenario

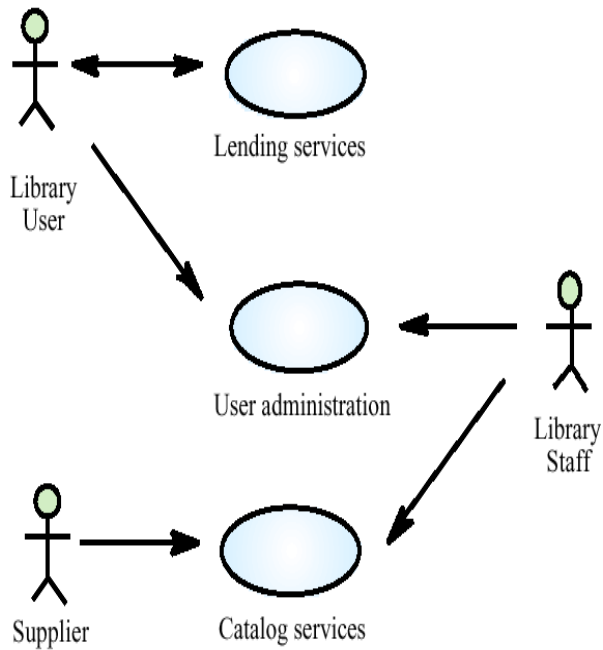
Developing a Use-Case

- What are the main tasks or functions that are performed by the actor?
- What system information will the actor acquire, produce or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

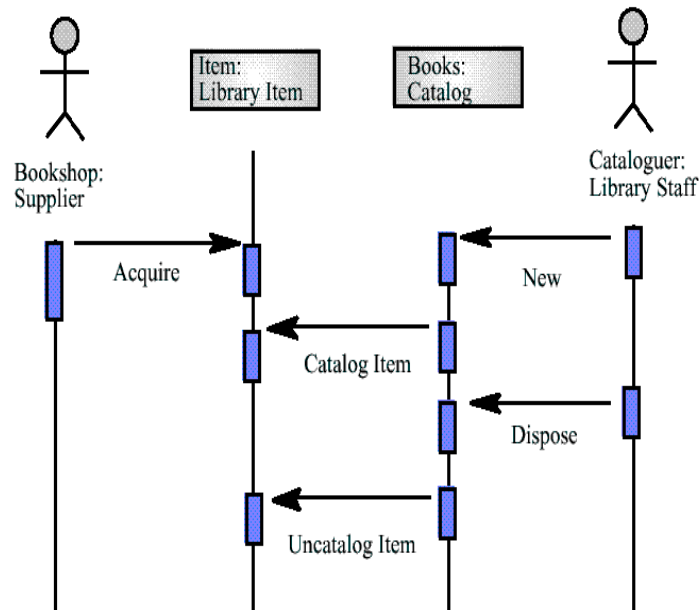
Lending use-case



Library Use-cases



Catalogue Management

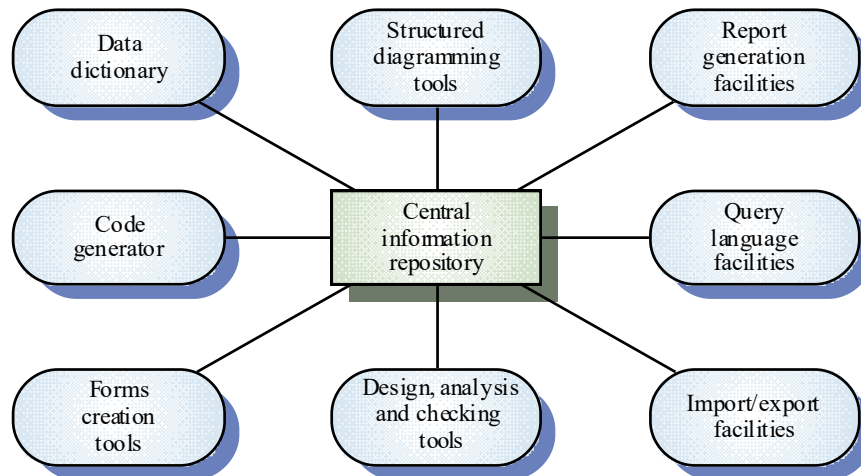


CASE Workbenches

- A coherent set of tools that is designed to support related software process activities such as analysis, design or testing.
- Analysis and design workbenches support system modelling during both requirements engineering and system design.

- These workbenches may support a specific design method or may provide support for a creating several different types of system model.

An analysis and design workbench



Analysis

Workbench Components

- Diagram editors
- Model analysis and checking tools
- Repository and associated query language
- Data dictionary
- Report definition and generation tools
- Forms definition tools
- Import/export translators
- Code generation tools